CS & IT ENGINEERING



Algorithms

Divide & Conquer



Lecture No.- 04

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Principle of Merging → V.V. Imp

Binary Search:

- Idea
- Recursive Algo
- Non-Recursive Algo
- TC analysis
- Details

Topics to be Covered



- Topic
- Topic
- Topic

Merge Sort — V.V. Imp

↳ (Merge Algo)



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

Telegram



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Merge Sort

(1) Merging Algo:- Summary

Sorted list(L_1) → m

Merge L_3: (n + m)

Sorted list(L_2) → n

[m ≤ number of element comparisons ≤ (m + n – 1)]

m ≤ n

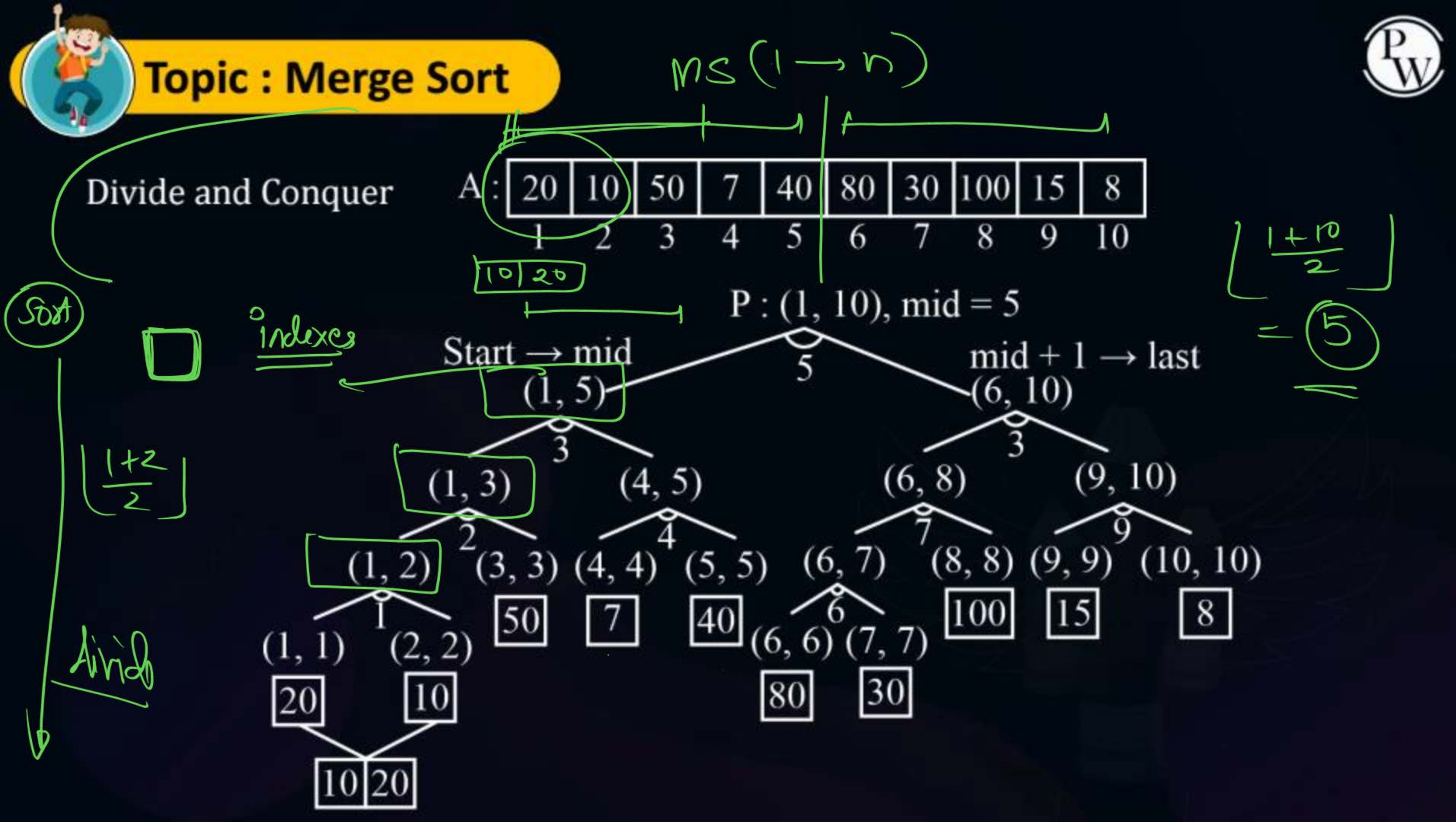
min

max

min(m, n)

Topic : Merge Sort

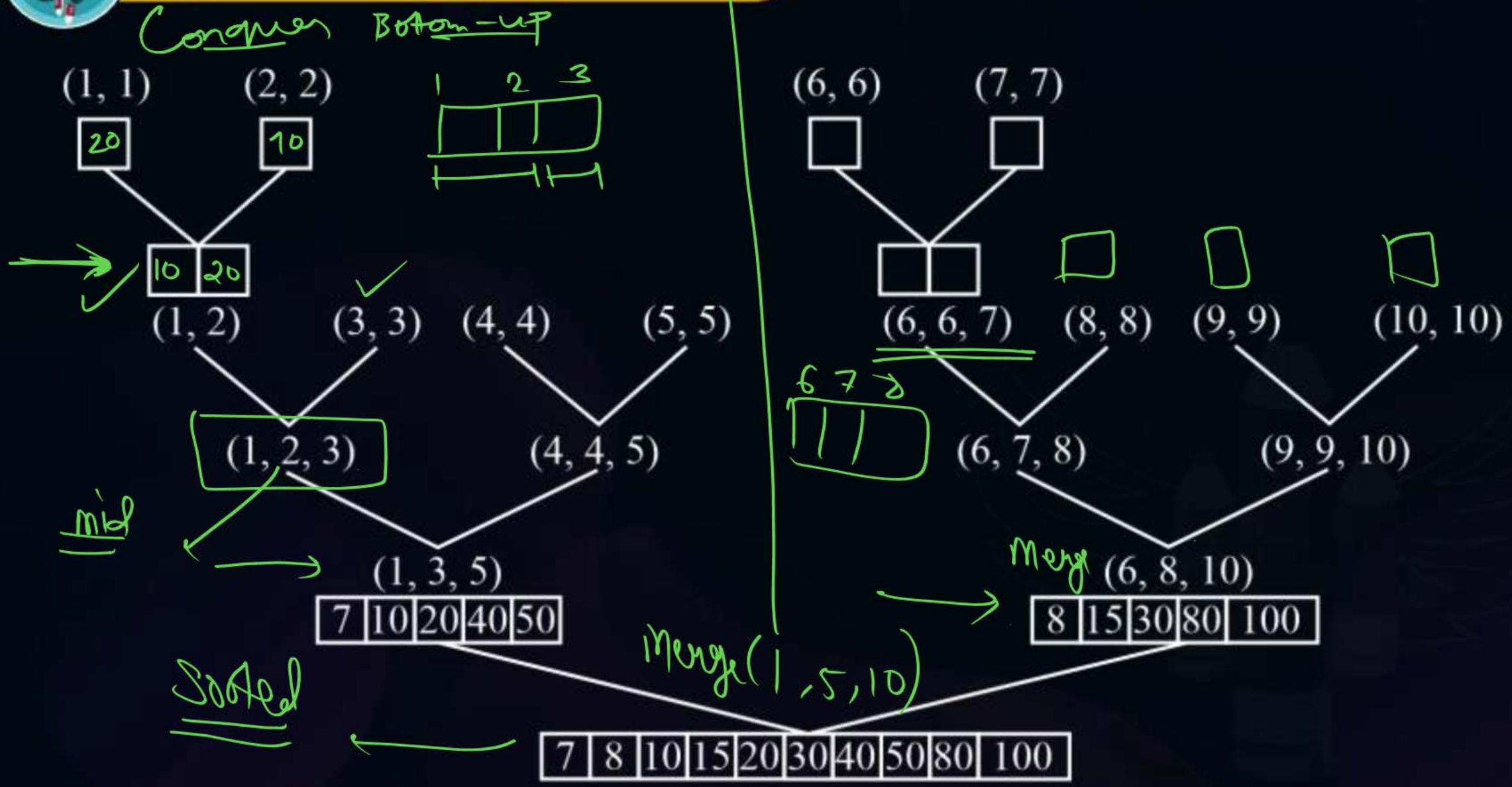
Divide and Conquer

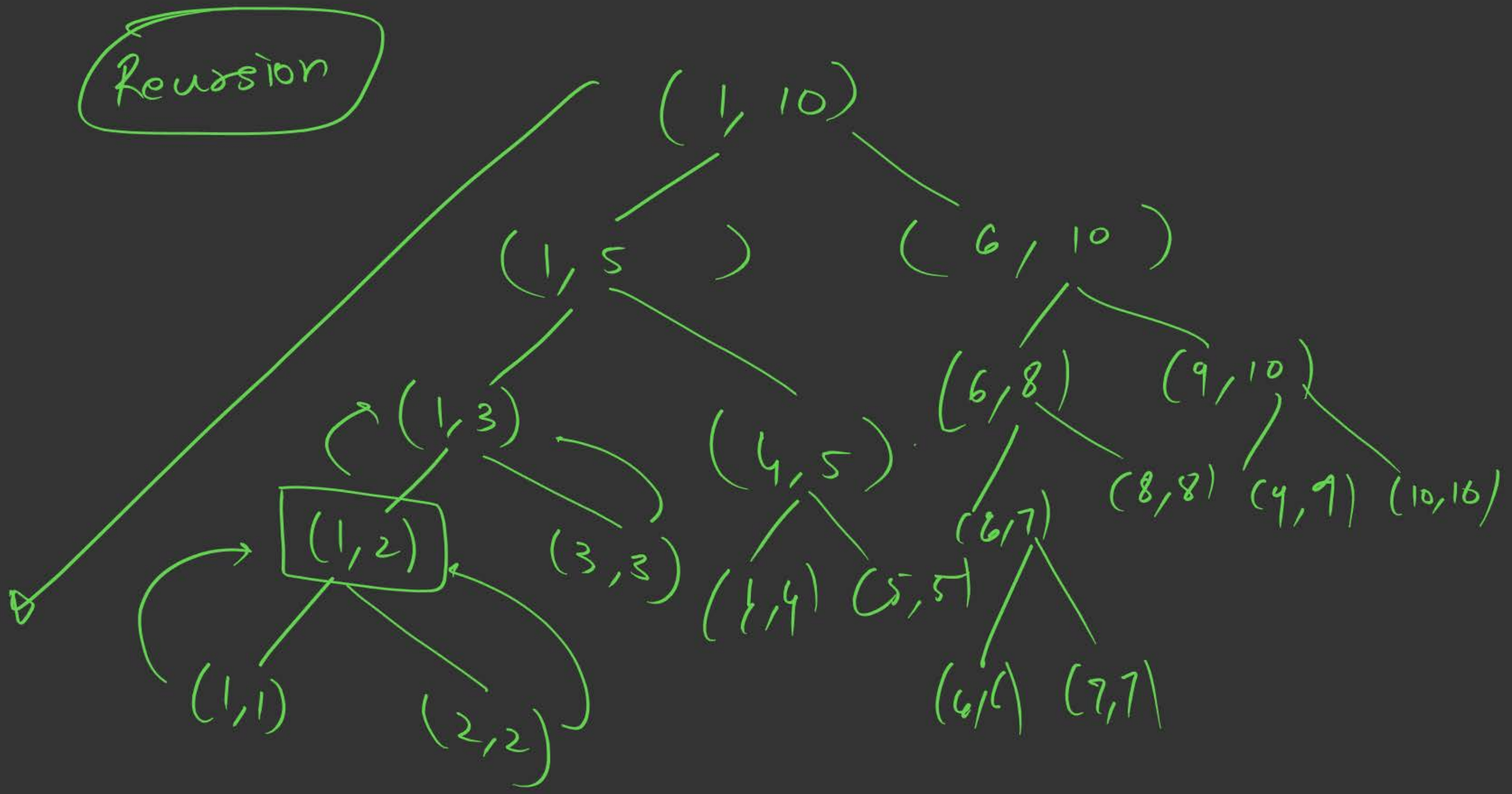




Topic : Sorting Algorithms



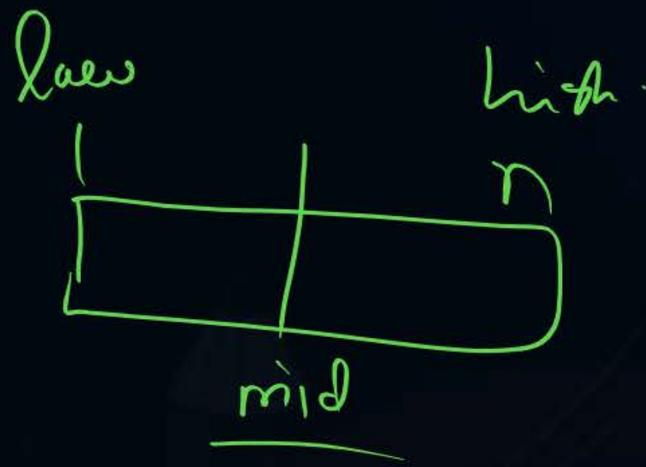



Recursion
(1, 10)
(1, 5)
(6, 10)
(1, 3)
(4, 5)
(6, 8)
(9, 10)
(1, 2)
(3, 3)
(4, 4)
(5, 5)
(6, 7)
(8, 8)
(9, 9)
(10, 10)
(1, 1)
(2, 2)
(6, 6)
(7, 7)



Topic : Divide and Conquer

```
1.  Algorithm MergeSort(low, high)
2.  //a[low: high] is a global array to be sorted.
3.  //Small(P) is true if there is only one element
4.  //to sort. In this case the list is already sorted.
5.  {
6.      if (low < high) then // If there are more than one element
7.      {
8.          // Divide P into subproblems.
9.          // Find where to split the set.
```





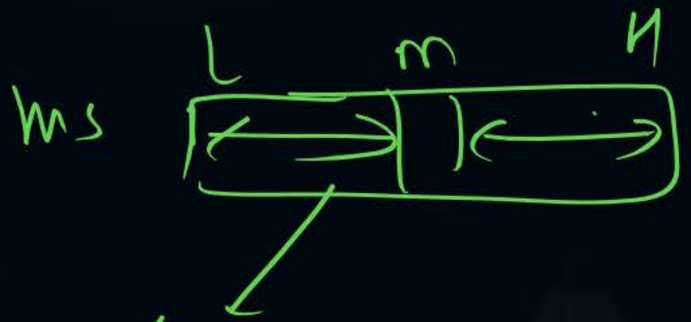
Topic : Divide and Conquer

```
10.          mid: = [(low + high)/2];
11.          // Solve the subproblems.
12.          MergeSort(low, mid);
13.          MergeSort(mid + 1, high);
14.          // Combine the solutions.
15.          Merge(low, mid, high);
16.     }
17. }
```

MergeSort(low, mid); → T(n/2)

MergeSort(mid + 1, high); → T(n/2)

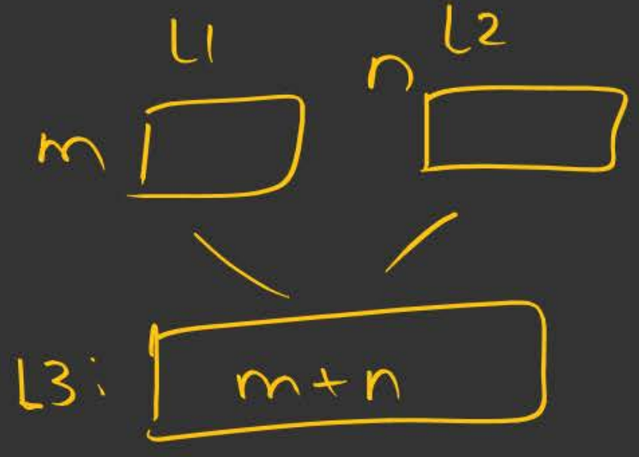
Merge(low, mid, high); → O(n)

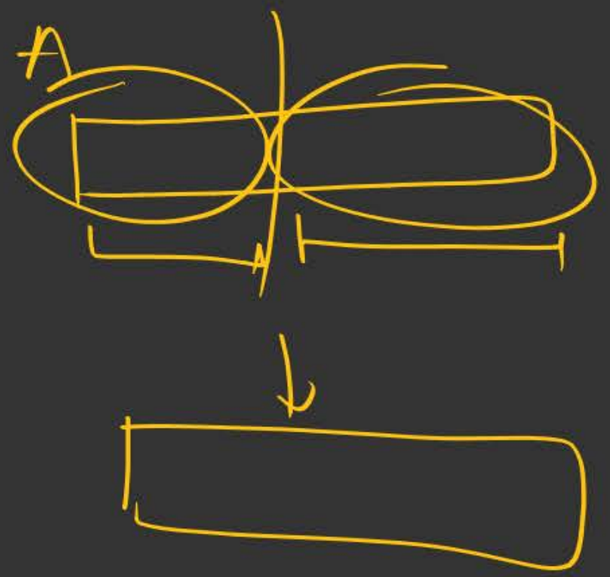


MS(L, m)

MS(m+1, H)

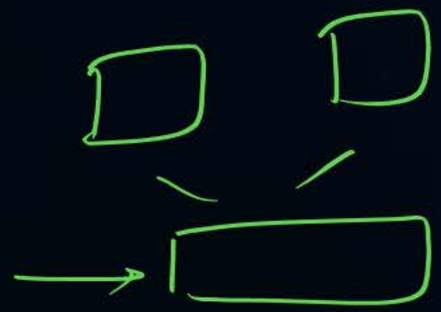
Merge(L, m, H)


L1
L2
m
n
L3:
m+n

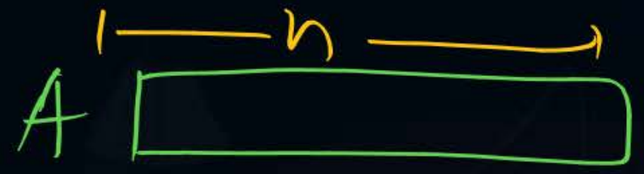

n

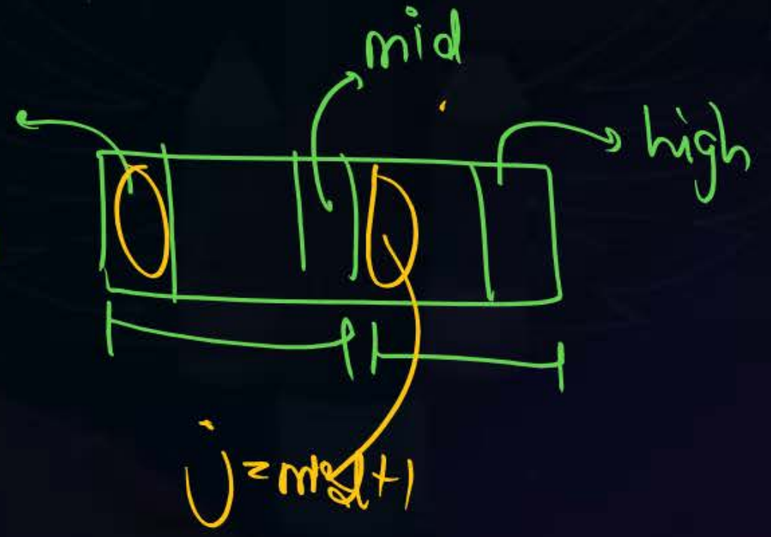
Topic : Divide and Conquer





```
1.  Algorithm Merge(low, mid, high)
2.  //a[low: high] is a global array containing two sorted
3.  //subsets in a [low: mid] and in a[mid +1: high]. The goal
4.  // is to merge these two sets into a single set residing
5.  // in a[low: high] b[ ] is an auxiliary global array.
6.  {
7.      h:= low; i := low; j := mid + 1;
8.      while ((h ≤ mid) and (j ≤ high)) do
9.      {
10.         if (a[h] ≤ [j]) then
```

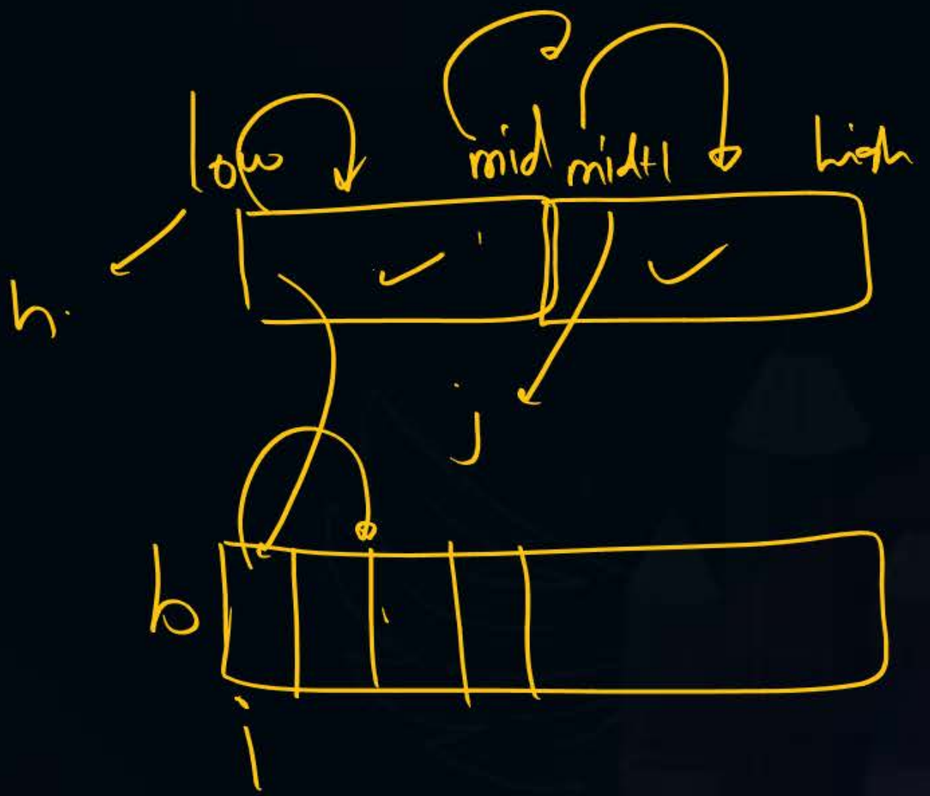






Topic : Divide and Conquer

```
11.      {
12.          b[i] := a[h];h := h + 1;
13.      }
14.  else
15.      {
16.          b[i] := a[j]; j = j + 1 ;
17.      }
18.      i := i + 1;
19.  }
20.  if (h > mid) then
```



first list is exhausted.

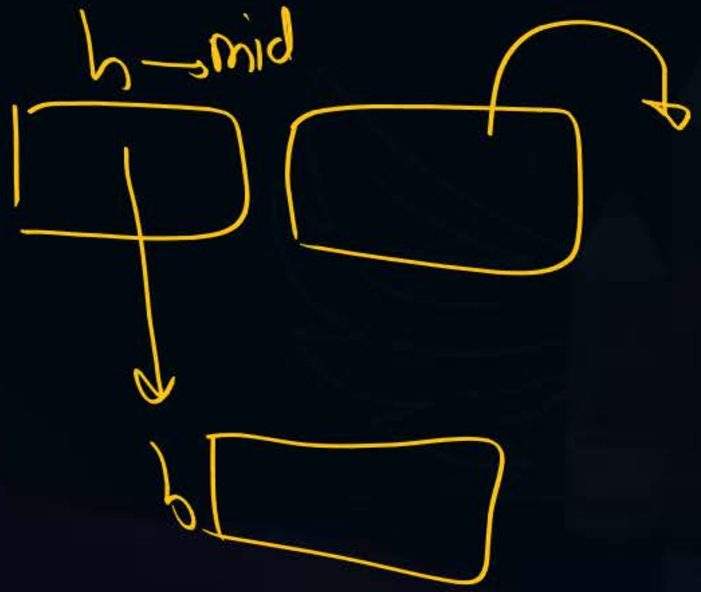


Topic : Divide and Conquer

```
21.  for k := j to high do
22.  {
23.      b[i] := a[k]; i := i + 1;
24.  }
25.  else
26.  for k := h to mid do
27.  {
28.      b[i] := a[k]; i := i + 1;
29.  }
```

// copy remaining elem from 2nd list

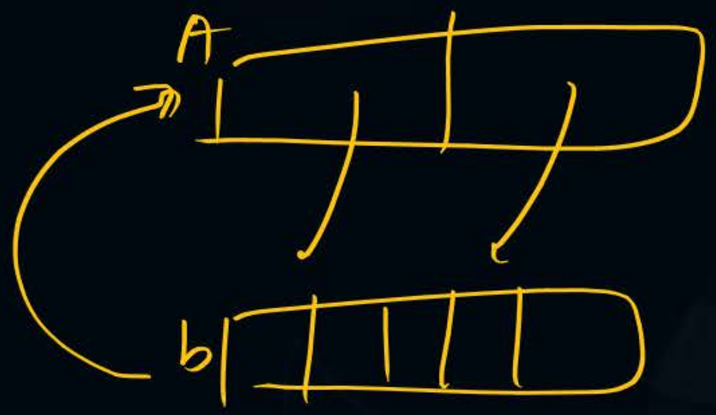






Topic : Divide and Conquer

```
30.     for k := low to high do a[k] := b[k];
31.  }
```



80%

20% → merge Sort

#Q. For merging two sorted lists of size m and n into a sorted list of size m + n, we require comparisons of

100% 90%

A O(m)

B O(n)

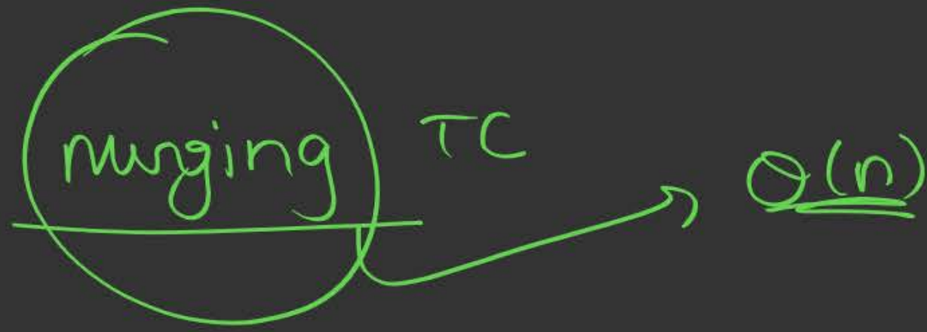
C O(m + n)

D O(log m + log n)

(m+n-1)

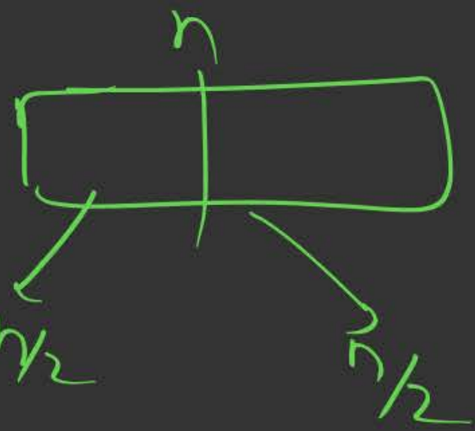
↓

O(m+n)



Best C $\longrightarrow O(n/2) = O(n)$

W.C $\longrightarrow O(n/2 + n/2 - 1) = O(n)$





Topic : Performance of Merge Sort

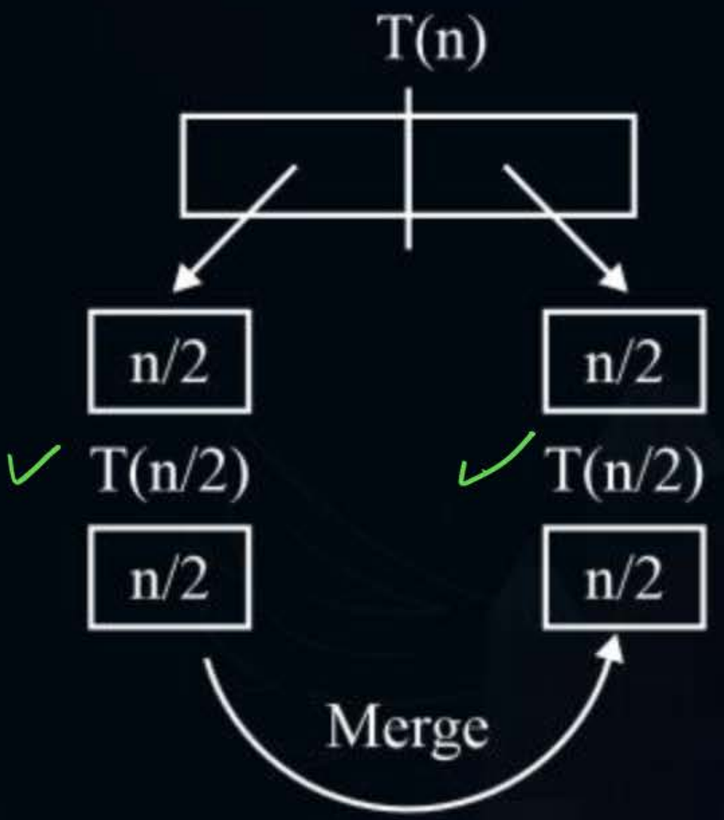
(1) Time Complexity:

Merging :- 2

Best Core :- $n/2 \Rightarrow O(n)$

Worst core :- $\left(\frac{n}{2} + \frac{n}{2} - 1\right)$

$= (n - 1) \rightarrow O(n)$





Topic : Performance of Merge Sort

Let T(n) represent time Complexity of Merge Sort on A[1 : n]

$T(n) = C, n = 1$ (Small problem)

$T(n) = 2T(n/2) + b \times n, n > 1$

$T(n) = T(n/2) + T(n/2) + O(n)$

$= 2T(n/2) + O(n)$

$= 2T(n/2) + n$

Most Accurate

$T(n) = 2T(n/2) + (n-1), n > 1$

Solution:-

$T(n) = 2T(n/2) + b * n, n > 1$

$T(n) = 2\left(2T\left(\frac{n}{2^2}\right) + \frac{bn}{2}\right) + b \times n$

$= 2^2T\left(\frac{n}{2^2}\right) + bn + bn$

$= 2^2T\left(\frac{n}{2^2}\right) + 2b \times n$

$= 2^3T\left(\frac{n}{2^3}\right) + 3b \times n$

Step3 :- $T(n) = \Theta(n\log_2{}^n)$ every case

General:-

$T(n) = 2^KT\left(\frac{n}{2^K}\right) + b \times K \times n$

For B.C,

$\frac{n}{2^K} = 1$

$n = 2^K \rightarrow K = \log_2{}^n$

$T(n) = n \times T(1) + b \times n \times \log_2{}^n$

$= n \times C + b \times n\log_2{}^n$

value of Rec

Topic : Merge Sort

TC

Best Case : $\Omega(n \log_2 n)$

Worst Case : $O(n \log_2 n)$

$\Theta(n \log 2n)$

SC

Max height of

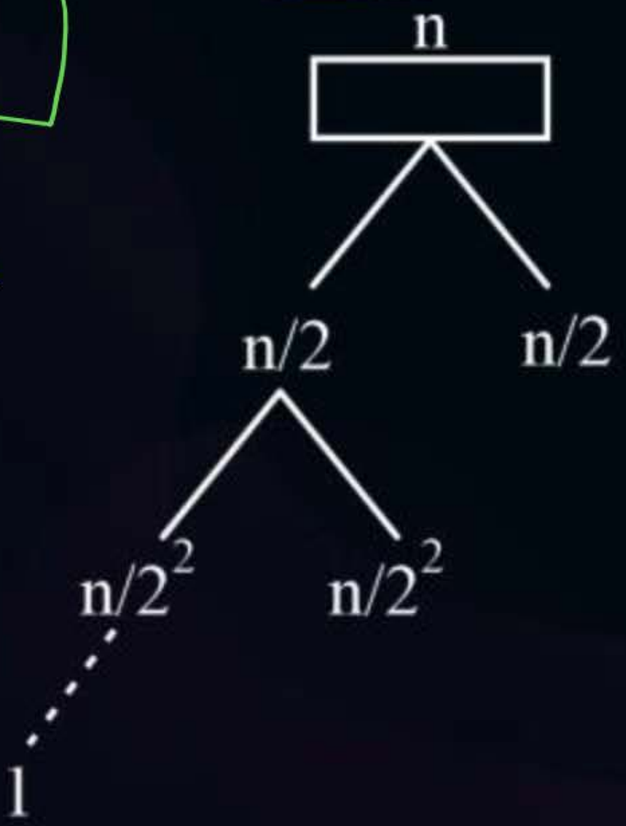
Recension ~~short~~ Stack

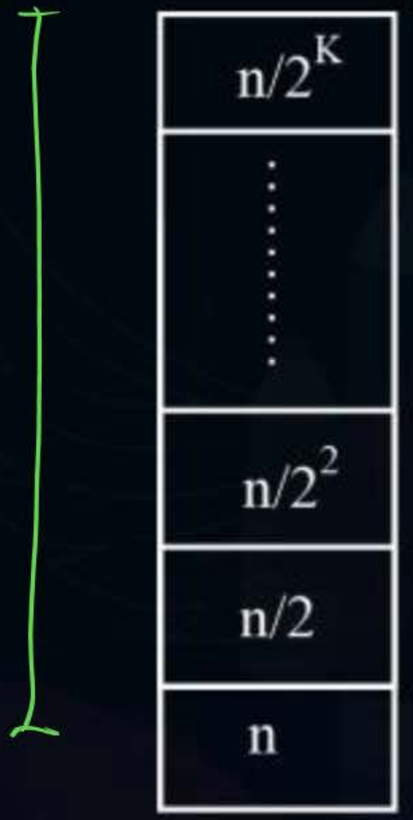
at any point during

~~Print during~~ MS(1, n)

Will be

$\log_2 n$





$n/2^K = 1$

$2^K = n$

$K = \log_2 n$



Topic : Space Complexity Analysis

(1) Temporary/Auxiliary

Array 'B' used for merging

$A = [1 \rightarrow n]$

$B = [1 \rightarrow n]$

$O(n)$

(2) Recursion Stack

$\Rightarrow \quad O(\log_2^n)$

Over all Space Complexity $= O(n + \log_2^n)$

$= O(n) \Rightarrow$ not inplace

Performance Analysis of Merge Sort → Summary

Time Complexity ⇒ O(n log n) {Time efficient}

But Space Complexity ⇒ O(n) {Space inefficient}

⇒ Merge Sort is stable. (not inplace)

Comparison

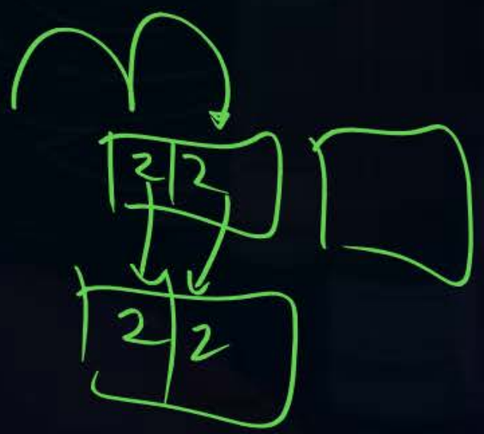
WC

Bubble $O(n^2)$

Insertion $O(n^2)$

Selection $O(n^2)$

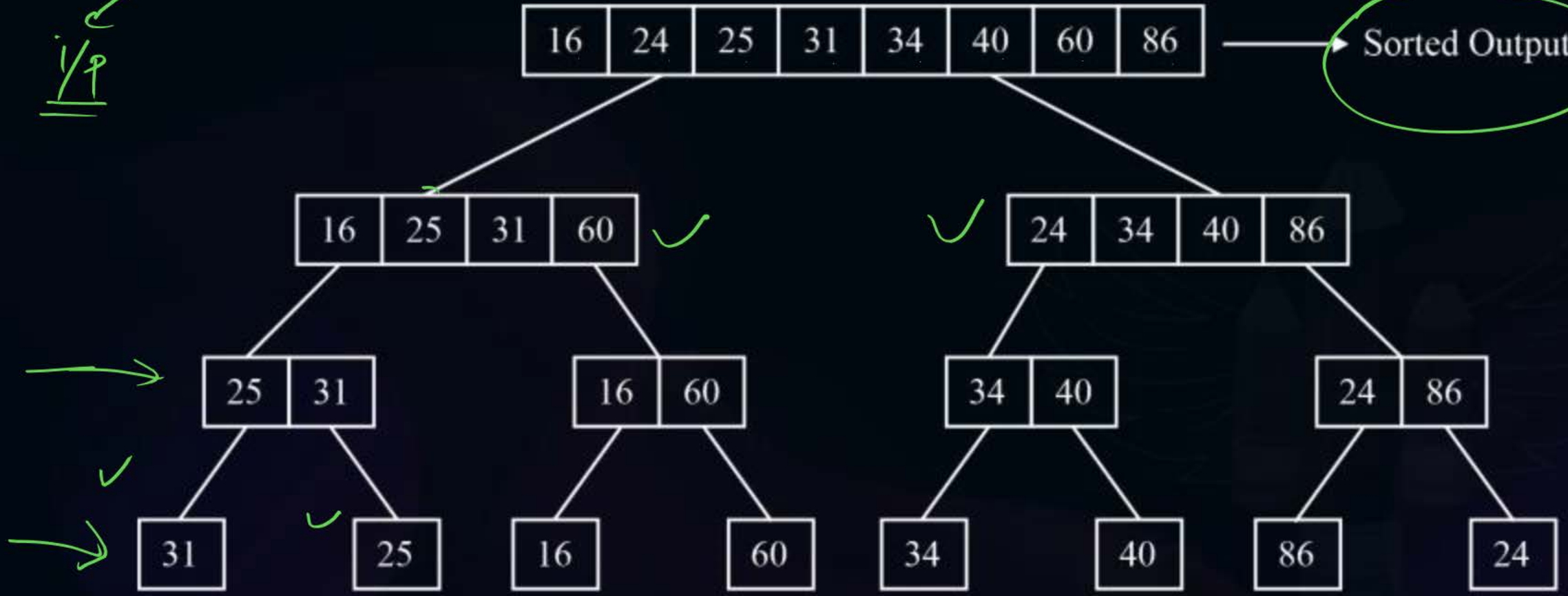
$n \log n \leq n^2$



Bonus

Version-2 :- Bottom-up Merge Sort / 2-way (Special logic)

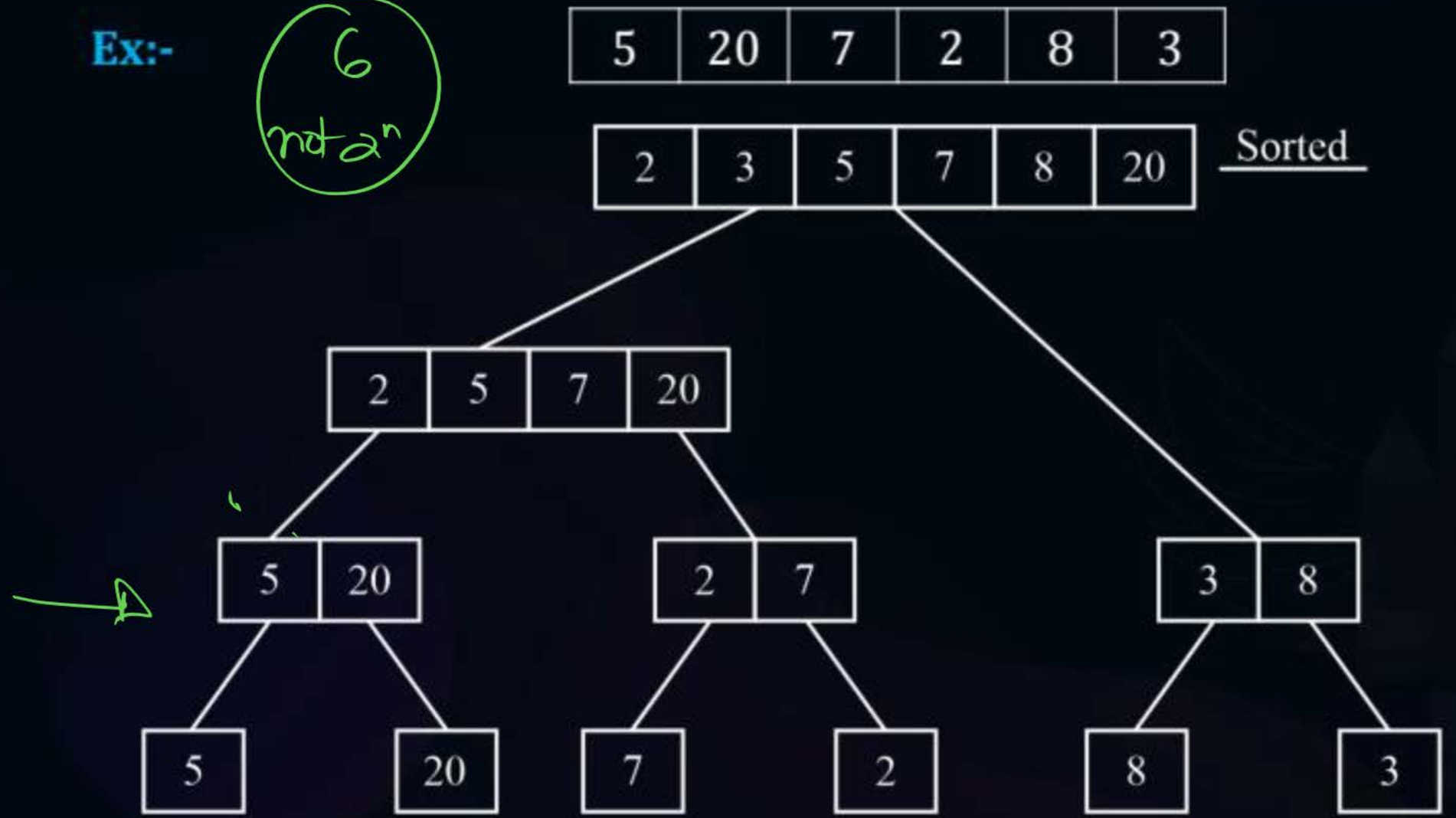
A = [31, 25, 16, 60, 34, 40, 86, 24]



If equal – pairing not possible

Ex:-

6
not an





2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Merge Sort

Merging



THANK - YOU